

Summary of Rust Runtime

1 Memory Management

Ownership System (Compile-Time): Rust's ownership system, including borrowing and lifetimes, is enforced entirely at compile time. This ensures memory safety and prevents data races without requiring a garbage collector or runtime checks.

Heap Allocation (Runtime): While the ownership system is compile-time, heap allocations (e.g., using `Box`, `Vec`, or `String`) occur at runtime. Rust's standard library provides the allocator, but the ownership system ensures that these allocations are safe and that deallocations happen automatically when values go out of scope.

2 Traits and Polymorphism

Static Dispatch (Compile-Time): Most of Rust's trait system is resolved at compile time. The compiler generates the appropriate code for trait methods where they are used.

Dynamic Dispatch (Runtime): When using trait objects (e.g., `dyn Trait`), Rust uses dynamic dispatch. This involves a vtable (virtual table) lookup at runtime to determine which method to call. This introduces some runtime overhead.

3 Threading

Concurrency Primitives: Rust's standard library provides concurrency primitives like threads, channels, and mutexes, which rely on underlying OS features and involve runtime support.

Thread Management: Rust's runtime manages thread creation, scheduling, joining, and synchronization, though the heavy lifting is done by the OS.

4 Standard Library

I/O Operations: Rust's standard library handles I/O operations, which involve system calls and runtime support to interface with the OS.

Error Handling: While Rust's `Result` and `Option` types are compile-time constructs, some runtime checks occur when handling these types.

5 Start-up and Clean-up

Initialization: Before the `main` function is called, Rust sets up a minimal runtime environment, including setting up the stack and initializing static variables.

Clean-up: After `main` returns, the runtime ensures any remaining resources are cleaned up, though this is minimal.

6 Language Features

Closures: Rust's closures can capture their environment. Depending on how they capture variables, there might be a small amount of runtime overhead to manage these captures.

7 Conclusion

Rust's runtime is intentionally minimal, focusing on essential tasks like panic handling, dynamic dispatch, threading, I/O, and some initialization and clean-up tasks. The bulk of Rust's safety guarantees are enforced at compile-time, leaving only a minimal runtime footprint.